

DATA SCIENCE CAPSTONE PROJECT

E-commerce Sales & Customer Insights Dashboard

Identifying revenue drivers and business opportunities across sales, customers, products, and geography

Prepared by Suvendu Mondal

Mindenious Edutech · Submission Deadline: 5 October 2026

Project Overview

Why this project matters

Objective

Analyze sales, customer behavior, products, and geography to identify revenue drivers and business opportunities.

Scope

Order, customer, product, and location data were combined into a single analytical view, cleaned, and used to derive KPIs, cohort trends, and a customer segmentation model — all surfaced through an interactive dashboard.

Dataset Snapshot

1,500

Customers

200

Products across 6 categories

40

Locations across 5 regions

19,803

Clean orders analyzed

Approach

A six-step pipeline from raw tables to a decision-ready dashboard

1

Combine

Merge order, customer, product, and location tables into one working dataset.

2

Clean

Remove duplicates and refund noise, impute missing values, standardize categories.

3

Build KPIs

Revenue, orders, AOV, repeat-customer rate, and margin.

4

Analyze Trends

Monthly trend and cohort analysis; RFM behavioral segmentation.

5

Find Patterns

Top products, weak categories, high-value customers, regional gaps.

6

Dashboard

Interactive, filterable dashboard with data-backed recommendations.

Data Cleaning

Raw order data was messy by design — here's what was fixed before analysis

579

Exact duplicate order lines removed

~1%

Orders with missing customer ID dropped (can't attribute revenue)

~2%

Missing quantities imputed using the median (robust to outliers)

6

Inconsistent category labels standardized (casing / whitespace)

4.2%

Refunded lines flagged and treated as negative revenue

Top-Line KPIs

Across the full analysis period

\$3.28M

Total Revenue

19,803

Total Orders

\$165.72

Avg. Order Value

22.65%

Margin

93.5%

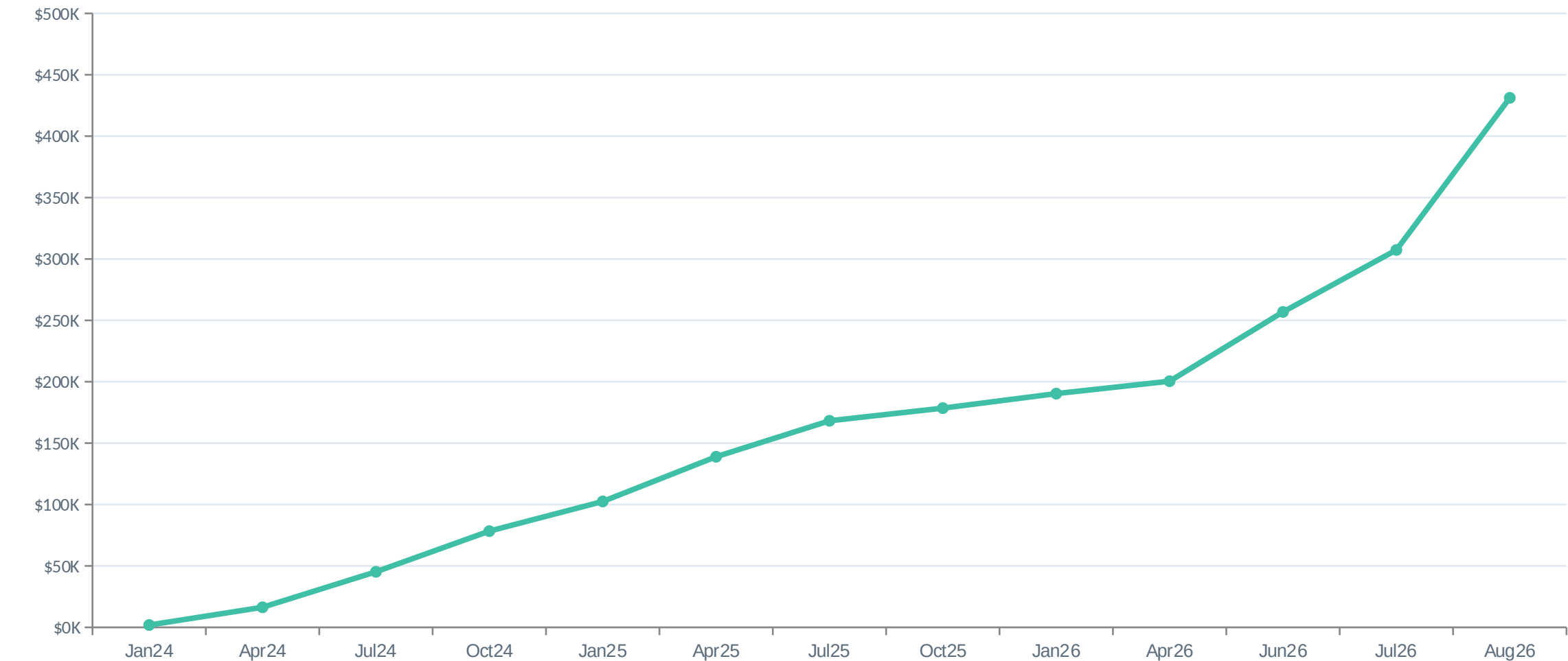
Repeat-Customer Rate

4.17%

Refund Rate

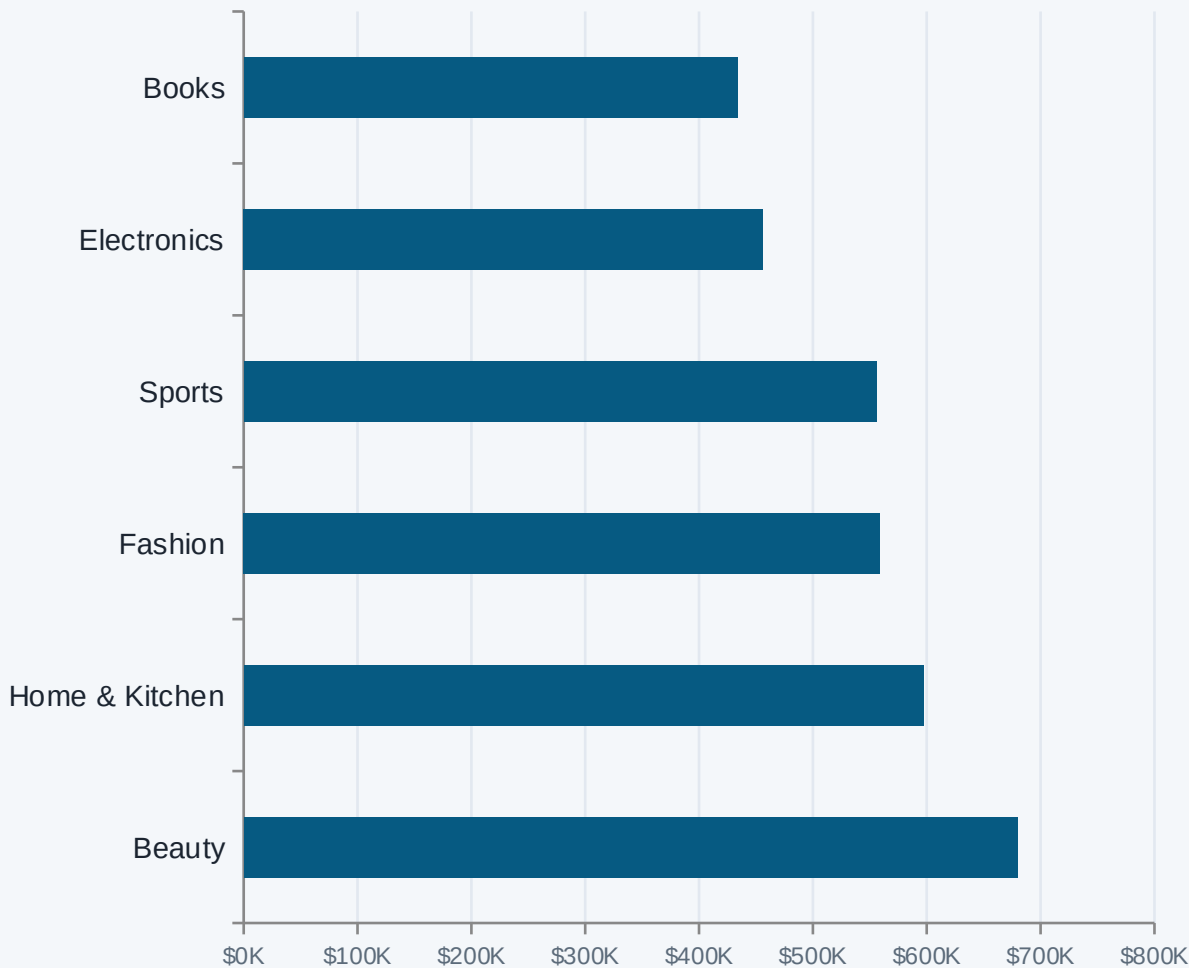
Monthly Revenue Trend

Consistent month-over-month growth from launch through Aug 2026



Category & Product Performance

Revenue by Category



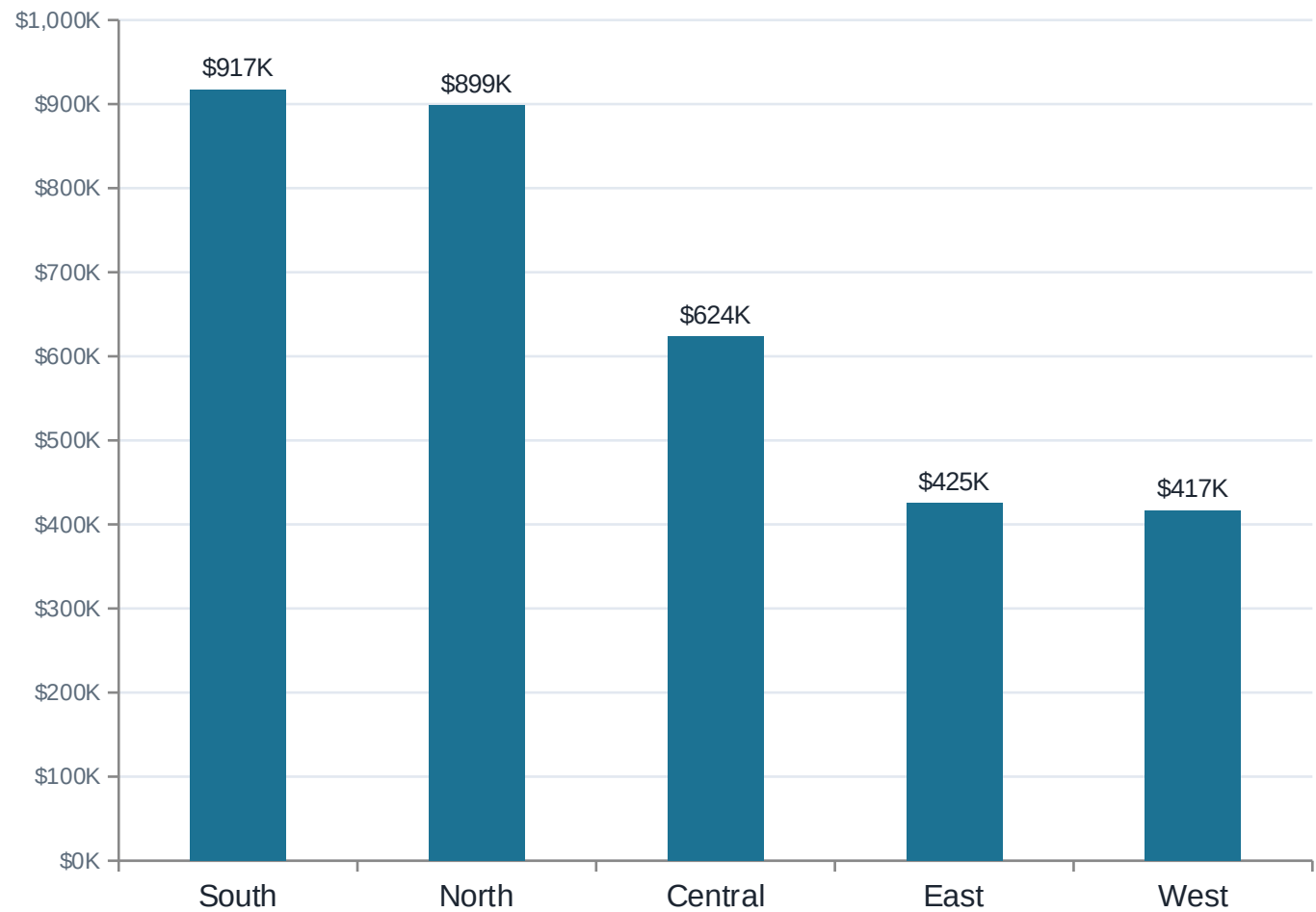
Top 5 Products by Revenue

1	Perfume Max	\$37,812
	Beauty	
2	Bluetooth Speaker Max	\$36,751
	Electronics	
3	Hair Dryer Lite	\$36,040
	Beauty	
4	Hair Dryer Basic	\$35,509
	Beauty	
5	Makeup Kit Plus	\$34,979
	Beauty	

Beauty products dominate the top sellers — 4 of the top 5 by revenue.

Regional Performance

South and North lead in revenue and customer count; East and West lag despite similar AOV



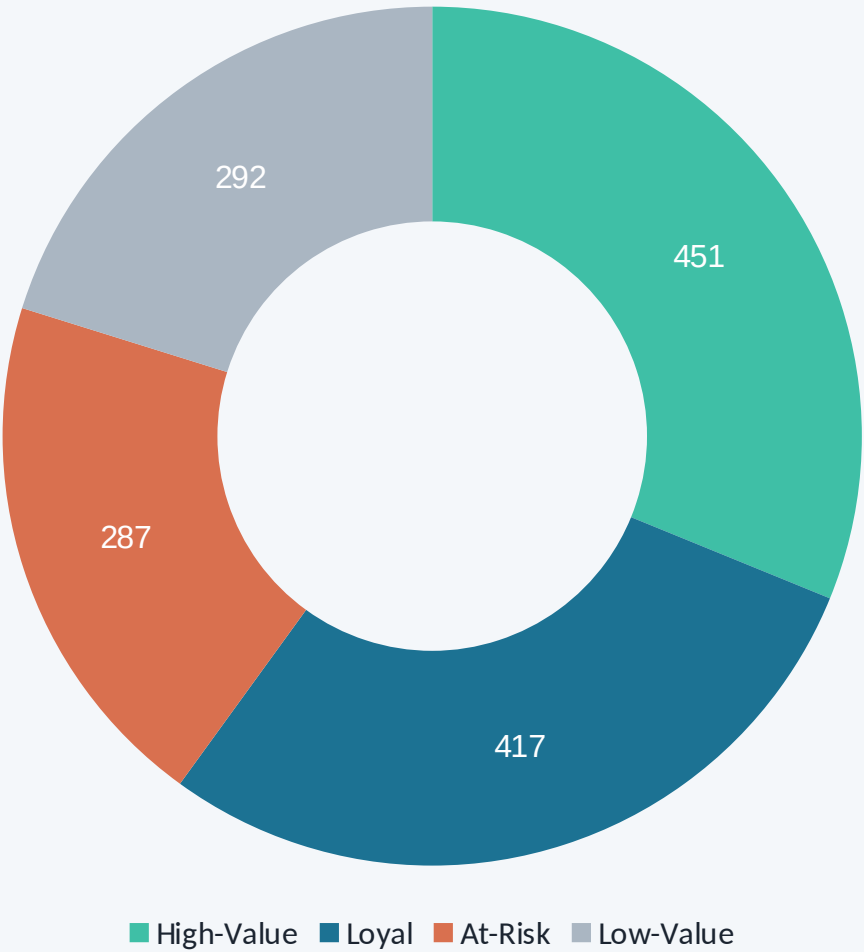
Regional AOV

South	\$166.86
North	\$168.46
Central	\$158.83
East	\$162.83
West	\$171.41

Takeaway: AOV is consistent across regions — the East/West gap is a reach problem, not a pricing one.

Customer Segmentation (RFM)

Recency, Frequency, and Monetary scoring across 1,447 active customers



What each segment means

- High-Value (451)**
Recent, frequent, big spenders — protect with loyalty perks.
- Loyal (417)**
Reliable repeat buyers — good candidates for upsell/cross-sell.
- At-Risk (287)**
Used to buy often, gone quiet recently — win-back campaign target.
- Low-Value (292)**
Infrequent, low spend — low-cost nurture or deprioritize.

Recommendations

Data-backed next steps

1

Double down on Beauty & Home/Kitchen

Highest revenue and healthy margins — prioritize inventory and ad spend here over lower-margin lines.

2

Win back At-Risk & Low-Value segments

~40% of customers fall into these RFM buckets — a targeted reactivation campaign can meaningfully lift repeat-purchase rate.

3

Close the East/West revenue gap

AOV is comparable to top regions — the shortfall looks like a marketing-reach problem worth testing, not a pricing one.

4

Audit refund-prone categories

Electronics and Books show the highest refund rates — review listings and product quality before scaling further.

Tech Stack

- ✓ Python (pandas, numpy) — data prep & analysis
- ✓ Streamlit + Plotly — interactive dashboard UI
- ✓ Git / GitHub — version control

Thank You

Suwendu Mondal · Data Science Capstone · Mindenious Edutech

APPENDIX

Full Code & Execution Output

Every code cell from the analysis notebook, run end-to-end, with its printed results and charts shown inline below.

E-commerce Sales & Customer Insights Dashboard

Data Science Capstone Project — Mindenious Edutech Prepared by Suvendu Mondal

This notebook walks through the full analysis pipeline used for the capstone brief:

1. Combine order, customer, product, and location tables
2. Clean duplicates, missing values, refunds, inconsistent categories
3. Build KPIs (revenue, orders, AOV, repeat-customer rate, margin)
4. Monthly trend + cohort-style RFM customer segmentation
5. Top products, category performance, and regional patterns
6. Data-backed recommendations

Runs standalone in Google Colab — no file uploads needed. A synthetic but realistic dataset is generated in Step 1 so the notebook is fully reproducible.

```
In [1]: # Install/import dependencies (Colab already has most of these)
!pip install -q faker --break-system-packages > /dev/null 2>&1 || pip install -q faker > /dev/null 2>&1

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.ticker as mticker
from faker import Faker
import random

random.seed(42)
np.random.seed(42)
fake = Faker()
Faker.seed(42)

plt.rcParams["figure.facecolor"] = "white"
plt.rcParams["axes.spines.top"] = False
plt.rcParams["axes.spines.right"] = False
```

Step 1 — Generate the raw dataset

Simulates customers, products, locations, and order lines (with intentional dirty data: duplicates, missing values, inconsistent category text) so the cleaning step is genuinely exercised.

```
In [2]: N_CUSTOMERS = 1500
N_PRODUCTS = 200
N_LOCATIONS = 40
N_ORDER_LINES = 20000
START_DATE = pd.Timestamp("2024-01-01")
END_DATE = pd.Timestamp("2026-08-31")

CATEGORIES = {
    "Electronics": ["Headphones", "Smartwatch", "Bluetooth Speaker", "Power Bank", "Webcam"],
    "Home & Kitchen": ["Air Fryer", "Blender", "Cookware Set", "Vacuum Cleaner", "Table Lamp"],
    "Fashion": ["Sneakers", "Jacket", "Handbag", "Sunglasses", "Wrist Watch"],
    "Beauty": ["Face Serum", "Perfume", "Hair Dryer", "Makeup Kit", "Sunscreen"],
    "Sports": ["Yoga Mat", "Dumbbell Set", "Cycling Helmet", "Running Shoes", "Water Bottle"],
    "Books": ["Fiction Novel", "Self-Help Book", "Cookbook", "Biography", "Comic Set"],
}

regions = ["North", "South", "East", "West", "Central"]

# Locations
locations_df = pd.DataFrame([
    {"location_id": f"LOC{i+1:03d}", "city": fake.city(), "state": fake.state(),
     "region": random.choice(regions)} for i in range(N_LOCATIONS)
])

# Customers
customers_df = pd.DataFrame([
    {"customer_id": f"CUST{i+1:05d}", "customer_name": fake.name(), "email": fake.email(),
     "signup_date": fake.date_between(start_date=START_DATE.date(), end_date=END_DATE.date()),
     "location_id": random.choice(locations_df["location_id"]),
     "segment": random.choices(["Regular", "Premium", "New"], weights=[0.55, 0.2, 0.25])[0]}
    for i in range(N_CUSTOMERS)
])

# Products
products = []
```

```

pid = 1
for cat, items in CATEGORIES.items():
    for item in items:
        for _ in range(N_PRODUCTS // (len(CATEGORIES) * len(items)) + 1):
            if pid > N_PRODUCTS:
                break
            base_cost = round(np.random.uniform(5, 150), 2)
            margin_pct = np.random.uniform(0.15, 0.55)
            products.append({
                "product_id": f"PRD{pid:04d}",
                "product_name": f"{item} {random.choice(['Pro', 'Lite', 'Max', 'Plus', 'Basic'])}",
                "category": random.choice([cat, cat.upper(), f" {cat}"]), # dirty on purpose
                "unit_cost": base_cost,
                "unit_price": round(base_cost * (1 + margin_pct), 2),
            })
            pid += 1
products_df = pd.DataFrame(products[:N_PRODUCTS])

# Orders (order-line level)
order_lines = []
customer_ids = customers_df["customer_id"].tolist()
product_ids = products_df["product_id"].tolist()
cust_signup = dict(zip(customers_df["customer_id"], pd.to_datetime(customers_df["signup_date"])))
purchase_weights = np.random.exponential(scale=1.5, size=len(customer_ids)) + 0.1

order_id_counter = 1
for i in range(N_ORDER_LINES):
    cust = random.choices(customer_ids, weights=purchase_weights, k=1)[0]
    signup = cust_signup[cust]
    order_date = fake.date_between(start_date=max(signup.date(), START_DATE.date()), end_date=END_DATE.date())
    prod = random.choice(product_ids)
    price = products_df.loc[products_df.product_id == prod, "unit_price"].values[0]
    qty = random.choices([1, 2, 3, 4, 5], weights=[0.6, 0.2, 0.1, 0.06, 0.04])[0]
    is_refund = random.random() < 0.04
    row = {
        "order_id": f"ORD{order_id_counter:06d}", "order_date": order_date,
        "customer_id": cust if random.random() > 0.01 else None,
        "product_id": prod, "quantity": qty if random.random() > 0.02 else None,
        "unit_price": price, "is_refunded": is_refund,
    }
    order_lines.append(row)
    order_id_counter += 1
    if random.random() < 0.03: # ~3% duplicate rows (dirty data)
        order_lines.append(row.copy())

orders_df = pd.DataFrame(order_lines)
print(f"locations: {len(locations_df)} | customers: {len(customers_df)} | "
      f"products: {len(products_df)} | orders (raw): {len(orders_df)}")
orders_df.head()

```

locations: 40 | customers: 1500 | products: 200 | orders (raw): 20579

Out[2]:

	order_id	order_date	customer_id	product_id	quantity	unit_price	is_refunded
0	ORD000001	2026-05-13	CUST00436	PRD0021	1.0	30.60	False
1	ORD000002	2026-05-31	CUST00397	PRD0165	1.0	82.24	False
2	ORD000003	2026-04-02	CUST00396	PRD0120	1.0	120.36	False
3	ORD000004	2026-02-09	CUST00125	PRD0172	1.0	39.85	False
4	ORD000005	2025-12-19	CUST01265	PRD0134	1.0	151.79	False

Step 2 — Clean & merge

Drop duplicates, drop orders with missing customer_id, impute missing quantity, standardize category text, and merge all four tables into one working dataframe.

In [3]:

```

before = len(orders_df)
orders_df = orders_df.drop_duplicates()
print(f"Dropped {before - len(orders_df)} exact duplicate rows")

orders_df = orders_df.dropna(subset=["customer_id"])
median_qty = orders_df["quantity"].median()
orders_df["quantity"] = orders_df["quantity"].fillna(median_qty)

products_df["category"] = products_df["category"].str.strip().str.title()

orders_df["line_amount"] = orders_df["quantity"] * orders_df["unit_price"]
orders_df["net_amount"] = np.where(orders_df["is_refunded"], -orders_df["line_amount"],
orders_df["line_amount"])

```

```

df = (orders_df
      .merge(customers_df, on="customer_id", how="left")
      .merge(products_df[["product_id", "product_name", "category", "unit_cost"]], on="product_id",
             how="left")
      .merge(locations_df, on="location_id", how="left"))

df["order_date"] = pd.to_datetime(df["order_date"])
df["signup_date"] = pd.to_datetime(df["signup_date"])
df["order_month"] = df["order_date"].values.astype("datetime64[M]")
df["cost_amount"] = np.where(df["is_refunded"], 0, df["quantity"] * df["unit_cost"])
df["margin_amount"] = df["net_amount"] - df["cost_amount"]

print(f"Clean, merged dataset: {len(df):,} rows")
df.head()

```

Dropped 579 exact duplicate rows
Clean, merged dataset: 19,803 rows

```
Out[3]:
```

	order_id	order_date	customer_id	product_id	quantity	unit_price	is_refunded	line_amount	net_amount	c
0	ORD000001	2026-05-13	CUST00436	PRD0021	1.0	30.60	False	30.60	30.60	
1	ORD000002	2026-05-31	CUST00397	PRD0165	1.0	82.24	False	82.24	82.24	
2	ORD000003	2026-04-02	CUST00396	PRD0120	1.0	120.36	False	120.36	120.36	
3	ORD000004	2026-02-09	CUST00125	PRD0172	1.0	39.85	False	39.85	39.85	
4	ORD000005	2025-12-19	CUST01265	PRD0134	1.0	151.79	False	151.79	151.79	

5 rows × 23 columns

Step 3 — Top-line KPIs

```

In [4]: total_revenue = df["net_amount"].sum()
total_orders = df["order_id"].nunique()
aov = total_revenue / total_orders
total_margin = df["margin_amount"].sum()
margin_pct = total_margin / total_revenue
repeat_rate = (df.groupby("customer_id")["order_id"].nunique() > 1).mean()
refund_rate = df["is_refunded"].mean()

kpis = {
    "Total Revenue": f"${total_revenue:,.0f}",
    "Total Orders": f"{total_orders:,}",
    "Avg Order Value": f"${aov:,.2f}",
    "Margin %": f"{margin_pct*100:.2f}%",
    "Repeat-Customer Rate": f"{repeat_rate*100:.1f}%",
    "Refund Rate": f"{refund_rate*100:.2f}%",
    "Active Customers": f"{df['customer_id'].nunique():,}",
}
for k, v in kpis.items():
    print(f"{k:<22}: {v}")

```

```

Total Revenue      : $3,281,785
Total Orders       : 19,803
Avg Order Value    : $165.72
Margin %           : 22.65%
Repeat-Customer Rate : 93.5%
Refund Rate        : 4.17%
Active Customers   : 1,447

```

Step 4 — Monthly revenue trend

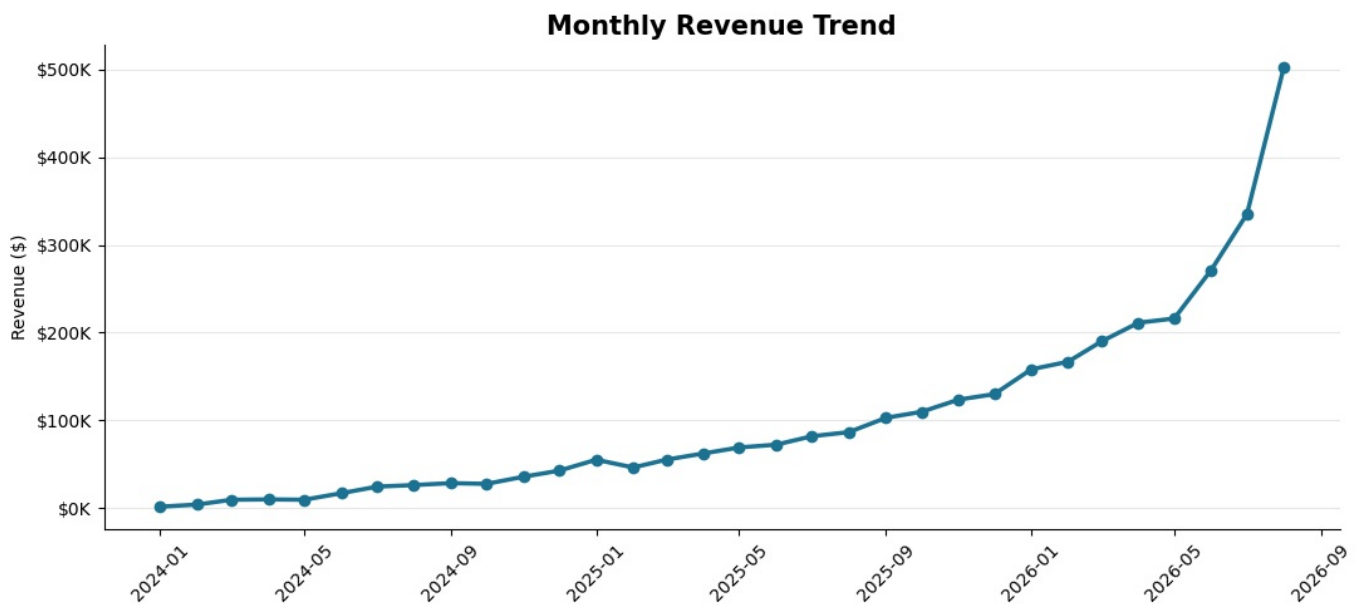
```

In [5]: monthly = (df.groupby("order_month")
                  .agg(revenue=("net_amount", "sum"), orders=("order_id", "nunique"))
                  .reset_index())

fig, ax = plt.subplots(figsize=(11, 5))
ax.plot(monthly["order_month"], monthly["revenue"], marker="o", color="#1C7293", linewidth=2.5)
ax.set_title("Monthly Revenue Trend", fontsize=15, fontweight="bold")
ax.set_ylabel("Revenue ($)")
ax.yaxis.set_major_formatter(mticker.FuncFormatter(lambda x, _: f"${x/1000:,.0f}K"))
ax.grid(axis="y", alpha=0.3)
plt.xticks(rotation=45)
plt.tight_layout()
plt.savefig("monthly_revenue_trend.png", dpi=150)

```

```
plt.show()
```

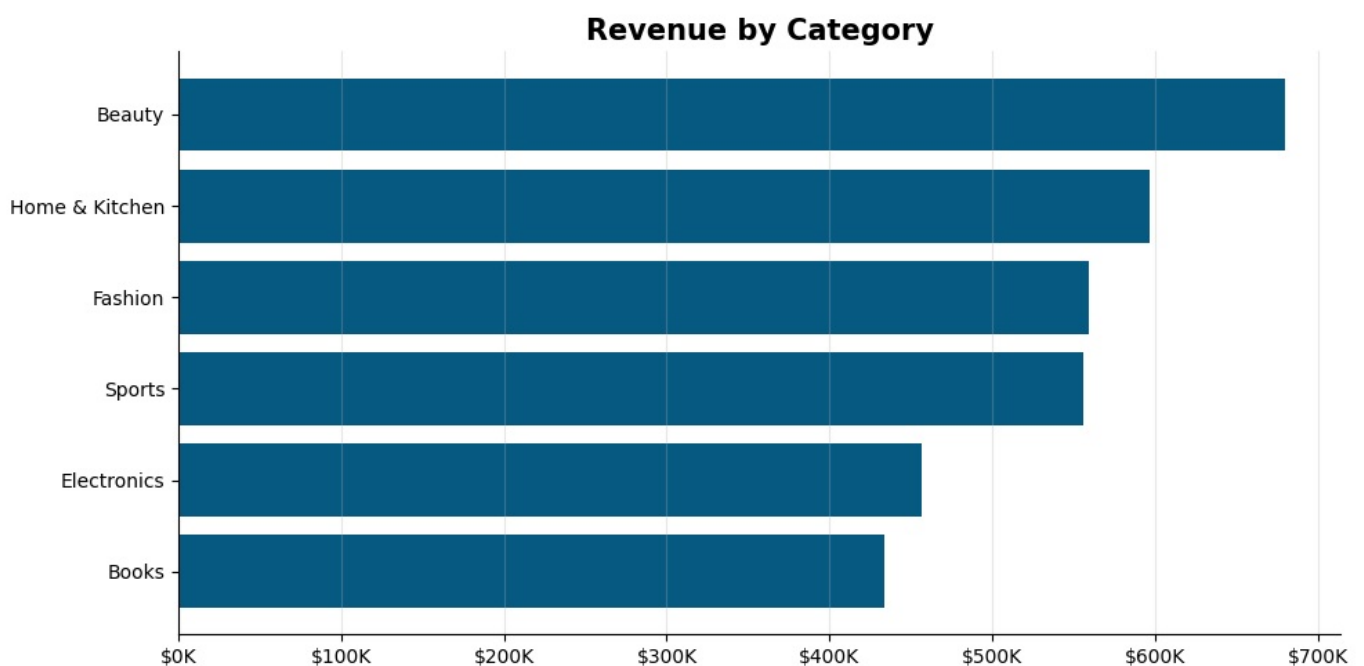


Step 5 — Category & product performance

```
In [6]: category_perf = (df.groupby("category")
    .agg(revenue=("net_amount", "sum"), margin=("margin_amount", "sum"),
        units_sold=("quantity", "sum"), refund_rate=("is_refunded", "mean"))
    .reset_index().sort_values("revenue", ascending=False))
category_perf["margin_pct"] = category_perf["margin"] / category_perf["revenue"] * 100

fig, ax = plt.subplots(figsize=(10, 5))
bars = ax.barh(category_perf["category"], category_perf["revenue"], color="#065A82")
ax.set_title("Revenue by Category", fontsize=15, fontweight="bold")
ax.xaxis.set_major_formatter(mticker.FuncFormatter(lambda x, _: f"${x/1000:,.0f}K"))
ax.invert_yaxis()
ax.grid(axis="x", alpha=0.3)
plt.tight_layout()
plt.savefig("category_performance.png", dpi=150)
plt.show()

category_perf
```

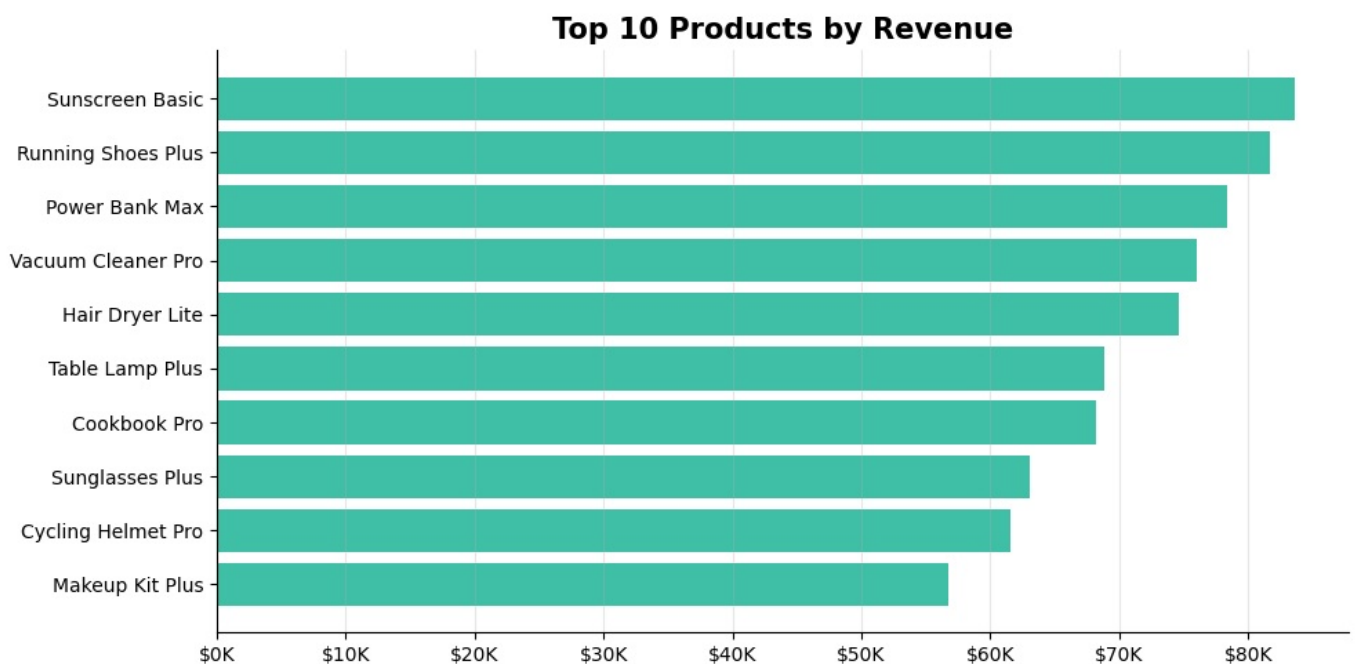


Out[6]:

	category	revenue	margin	units_sold	refund_rate	margin_pct
0	Beauty	679796.65	165680.70	5956.0	0.040941	24.372097
4	Home & Kitchen	597004.36	139042.29	5994.0	0.036879	23.289996
3	Fashion	559169.45	117606.32	5833.0	0.040481	21.032322
5	Sports	555764.37	116995.75	6160.0	0.043069	21.051323
2	Electronics	456381.95	103384.73	5853.0	0.044947	22.653115
1	Books	433668.70	100553.20	4455.0	0.044533	23.186640

```
In [7]: top_products = (df.groupby(["product_name", "category"])["net_amount"].sum()
        .sort_values(ascending=False).head(10).reset_index())

fig, ax = plt.subplots(figsize=(10, 5))
ax.barh(top_products["product_name"], top_products["net_amount"], color="#3FBFA6")
ax.set_title("Top 10 Products by Revenue", fontsize=15, fontweight="bold")
ax.xaxis.set_major_formatter(mticker.FuncFormatter(lambda x, _: f"${x/1000:,.0f}K"))
ax.invert_yaxis()
ax.grid(axis="x", alpha=0.3)
plt.tight_layout()
plt.savefig("top_products.png", dpi=150)
plt.show()
```



Step 6 — Regional patterns

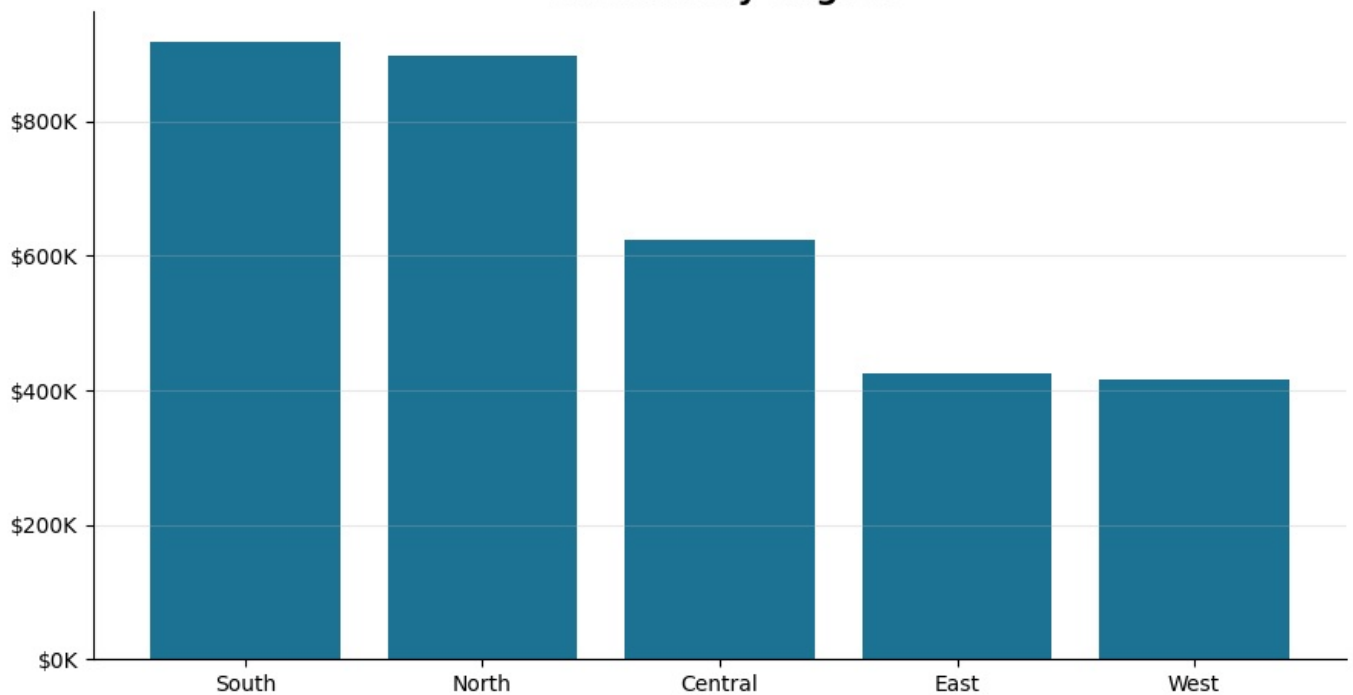
```
In [8]: region_perf = (df.groupby("region")
        .agg(revenue=("net_amount", "sum"), orders=("order_id", "nunique"),
            customers=("customer_id", "nunique"))
        .reset_index().sort_values("revenue", ascending=False))

region_perf["aov"] = region_perf["revenue"] / region_perf["orders"]

fig, ax = plt.subplots(figsize=(9, 5))
ax.bar(region_perf["region"], region_perf["revenue"], color="#1C7293")
ax.set_title("Revenue by Region", fontsize=15, fontweight="bold")
ax.yaxis.set_major_formatter(mticker.FuncFormatter(lambda x, _: f"${x/1000:,.0f}K"))
ax.grid(axis="y", alpha=0.3)
plt.tight_layout()
plt.savefig("regional_performance.png", dpi=150)
plt.show()

region_perf
```

Revenue by Region



Out[8]:

	region	revenue	orders	customers	aov
3	South	917374.25	5498	395	166.855993
2	North	898543.40	5334	417	168.455831
0	Central	624036.17	3929	251	158.828244
1	East	425137.51	2611	214	162.825550
4	West	416694.15	2431	170	171.408536

Step 7 — Customer segmentation (RFM)

```
In [9]: snapshot_date = df["order_date"].max() + pd.Timedelta(days=1)
rfm = (df.groupby("customer_id")
        .agg(recency=("order_date", lambda x: (snapshot_date - x.max()).days),
             frequency=("order_id", "nunique"),
             monetary=("net_amount", "sum"))
        .reset_index())

rfm["r_score"] = pd.qcut(rfm["recency"], 4, labels=[4, 3, 2, 1]).astype(int)
rfm["f_score"] = pd.qcut(rfm["frequency"].rank(method="first"), 4, labels=[1, 2, 3, 4]).astype(int)
rfm["m_score"] = pd.qcut(rfm["monetary"], 4, labels=[1, 2, 3, 4]).astype(int)
rfm["rfm_score"] = rfm["r_score"] + rfm["f_score"] + rfm["m_score"]

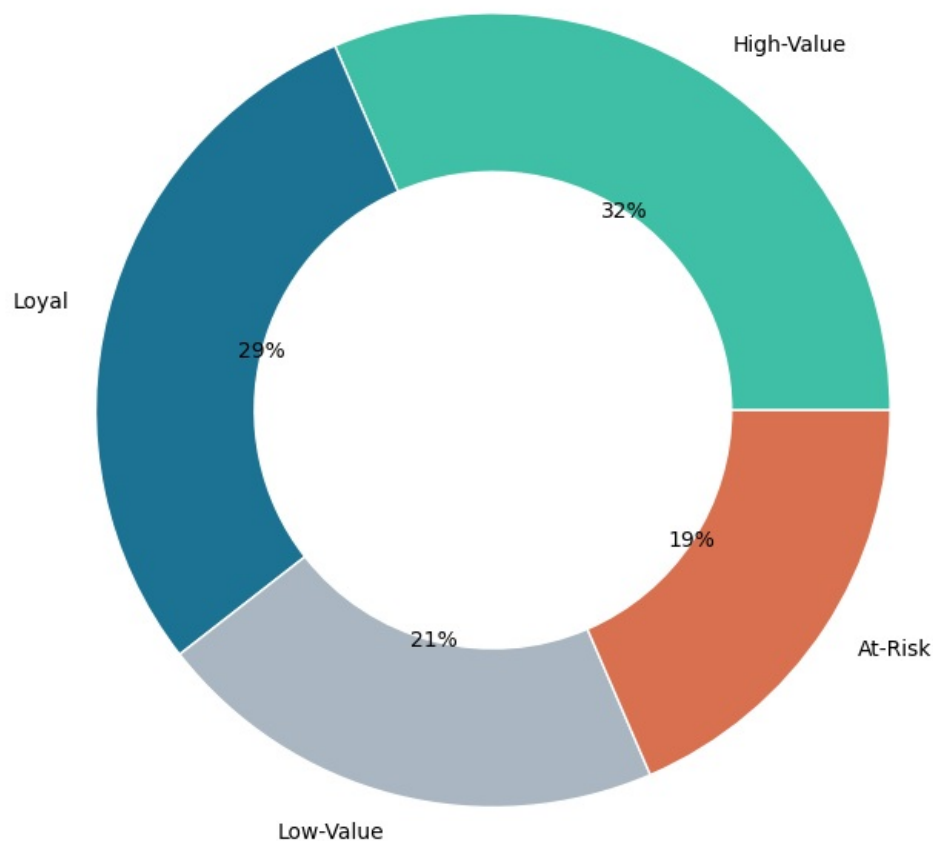
def segment_customer(row):
    if row["rfm_score"] >= 10: return "High-Value"
    elif row["rfm_score"] >= 7: return "Loyal"
    elif row["rfm_score"] >= 5: return "At-Risk"
    else: return "Low-Value"

rfm["segment"] = rfm.apply(segment_customer, axis=1)
seg_counts = rfm["segment"].value_counts()

fig, ax = plt.subplots(figsize=(7, 7))
colors = {"High-Value": "#3FBFA6", "Loyal": "#1C7293", "At-Risk": "#D9704F", "Low-Value": "#AAB6C2"}
ax.pie(seg_counts.values, labels=seg_counts.index, autopct="%1.0f%%",
       colors=[colors[s] for s in seg_counts.index],
       wedgeprops={"width": 0.4, "edgecolor": "white"})
ax.set_title("Customer Segments (RFM)", fontsize=15, fontweight="bold")
plt.tight_layout()
plt.savefig("rfm_segments.png", dpi=150)
plt.show()

seg_counts
```

Customer Segments (RFM)



```
Out[9]: segment
High-Value    456
Loyal          420
Low-Value     303
At-Risk        268
Name: count, dtype: int64
```

💡 Recommendations

- **Double down on Beauty & Home/Kitchen** — highest revenue and healthy margin; prioritize inventory and ad spend here over lower-margin categories.
- **Win back At-Risk & Low-Value segments** — roughly 40% of customers fall into these RFM buckets; a targeted reactivation campaign can meaningfully lift repeat-purchase rate.
- **Close the East/West revenue gap** — AOV is comparable to top regions, so the shortfall looks like a marketing-reach problem worth testing, not a pricing one.
- **Audit refund-prone categories** — Electronics and Books show the highest refund rates; worth a listing/quality review before scaling further.
- **Protect AOV** — consider a small free-shipping threshold just above the current AOV (~\$165) to nudge basket sizes up.